

TECHNICAL DOCUMENT

ECOM-OPS-8T

E-commerce Operations System "8tasks"

Technical Operations Manual

Architectural and Operational Specification

Document Code: DOC-ECOM-OPS-8T-001
Version: 1.0
Date: February 03, 2026
Classification: INTERNAL - COMPANY USE ONLY
Status: APPROVED
KNOWLEDGE TRANSFER DOCUMENT

Revision History

Version	Date	Author	Change Description
1.0	02/03/2026	Engineering Team	Initial document release

Approval Register

Role	Name	Date	Signature
Prepared by			
Reviewed by			
Approved by			

Table of Contents

1. System Introduction

1.1 Document Purpose

This document constitutes the complete technical operational specification for the ECOM-OPS-8T (E-commerce Operations System "8tasks") system. Its primary purpose is to provide a detailed description of the software architecture, configuration parameters, temporal dependencies, and operational dynamics of the system, in order to support development, maintenance, testing, and business process management activities.

This document has been prepared with particular attention to knowledge transfer, thus representing a fundamental resource for onboarding new technical personnel and ensuring operational continuity of the system.

1.2 Scope of Application

The ECOM-OPS-8T system represents a complex software architecture specifically designed for the management and orchestration of a multi-stage e-commerce order processing pipeline. The system was conceived to operate in a high-throughput environment where reliability, timing precision, and efficient use of computational resources are fundamental requirements for business success.

The system architecture is based on a distributed execution model that coordinates multiple specialized services, each responsible for specific functional domains like payment, inventory, and shipping. These services orchestrate the execution of interdependent tasks that must comply with strict timing constraints and a complex network of precedence relationships.

1.3 Intended Audience

This document is intended for: software engineers responsible for system development and maintenance, operations managers responsible for process monitoring and control, quality assurance teams for verification and validation

activities, training personnel for knowledge transfer programs, and technical management for project activity supervision.

2. Global Configuration and Operational Parameters

2.1 Operational Time Window

The ECOM-OPS-8T system operates within a well-defined time window extending from the initial instant, identified as time zero, up to time unit one hundred fifty. This operational window of one hundred fifty time units represents the maximum period during which all scheduled operations for a given transaction cycle must be started, executed, and completed.

The choice of this specific duration is closely linked to the Service Level Agreements (SLAs) for order fulfillment, ensuring that all critical business operations can be completed within the constraints imposed by customer expectations and logistical timelines.

2.2 System Parameters

Parameter	Value	Description
System Name	8tasks	Unique instance identifier
Operational Window (t_max)	150 t.u.	Maximum operational cycle duration
Max Resources (r_max)	110	Simultaneous processing units
Active Services	5	Number of service modules
Total Tasks	8	Number of operational tasks

The availability of one hundred ten resources allows for a massive degree of parallelism in task execution, enabling the system to effectively manage a high volume of concurrent order processing workflows. This high resource capacity is essential for meeting performance targets but also necessitates a sophisticated scheduling and dependency management model to prevent bottlenecks and ensure process integrity.

3. Modular Service Architecture

3.1 Architecture Overview

The ECOM-OPS-8T system architecture is structured into five main services, each designed to manage a specific functional domain of the e-commerce pipeline. This modular organization enables a clear separation of responsibilities and facilitates both maintenance and system evolution over time.

ID	Service	Primary Responsibility
1	Payment_Service	Manages payment validation and authorization
2	Inventory_Service	Handles stock checking, warehouse allocation, and quality control
3	Order_Service	Manages order confirmation and final processing
4	Shipping_Service	Responsible for shipping cost calculation and logistics preparation
5	Notification_Service	Manages all customer-facing communications

3.2 Detailed Service Descriptions

3.2.1 Payment_Service (ID: 1)

The Payment_Service is the entry point for the entire operational workflow. This service has the critical responsibility of managing the financial transaction through a single essential task: Payment_Validation (PAY_VAL). The correct and timely execution of this task is a prerequisite for a large portion of the downstream process. The task has a duration of 5 time units, requires 4 computational resources, and can run up to 4 concurrent instances.

3.2.2 Inventory_Service (ID: 2)

The Inventory_Service is responsible for the complete lifecycle of physical product management for an order. It coordinates three distinct tasks: Inventory_Check (INV_CHK), Warehouse_Allocation (WH_ALLOC), and Quality_Check (QC_CHK). This service ensures that stock is available, allocated from the correct warehouse, and meets quality standards. INV_CHK and WH_ALLOC are repetitive tasks (4 executions each) designed for thorough and iterative stock management, while QC_CHK is a single-run task. This service demonstrates a wide range of resource requirements, from 5 units for INV_CHK to just 1 for the other tasks.

3.2.3 Order_Service (ID: 3)

The Order_Service manages the core state of the customer order. It orchestrates two critical tasks: Order_Confirmation (ORD_CONF) and Final_Processing (FINAL_PROC). This service acts as a major convergence point in the workflow, with Order_Confirmation depending on the successful completion of payment, inventory, and shipping calculations. Final_Processing is a repetitive task (4 executions) requiring 5 resources, representing a significant computational load at the end of the pipeline.

3.2.4 Shipping_Service (ID: 4)

The Shipping_Service is dedicated to logistical calculations. It executes a single, resource-intensive repetitive task: Shipping_Calculation (SHIP_CALC). This task runs for 10 time units, repeats 3 times (for a total of 4 executions), and requires 3 resources per instance. Its role is to determine shipping costs and methods, a crucial input for the final order confirmation.

3.2.5 Notification_Service (ID: 5)

The Notification_Service handles all communications with the customer. It contains one primary task, Customer_Notification (CUST_NOTIF), which is responsible for sending status updates, confirmations, and tracking information. This task is lightweight, requiring only 1 resource for its 4 time unit duration, and is configured for two executions (1 initial + 1 repeat). It is triggered after the order has been successfully confirmed.

4. Detailed Description of Operational Tasks

4.1 Complete Task Registry

ID	Task Code	Service	Duration	Resources	Max Concur.	Rep
1	PAY_VAL	Payment_Service	5	4	4	0/0
2	INV_CHK	Inventory_Service	4	5	5	3/0
3	SHIP_CALC	Shipping_Service	10	3	3	3/0
4	ORD_CONF	Order_Service	8	2	2	0/0
5	CUST_NOTIF	Notification_Service	4	1	2	1/0

ID	Task Code	Service	Duration	Resources	Max Concur.	Rep
6	WH_ALLOC	Inventory_Service	5	1	5	3/0
7	QC_CHK	Inventory_Service	9	1	2	0/0
8	FINAL_PROC	Order_Service	6	5	5	3/0

Legend: Duration = time units; Resources = required computational units; Max Concur. = maximum concurrent executions; Rep = repetitions (rc/rd format where rc = repeat count, rd = repeat delay)

A notable characteristic of this system is the heterogeneous resource allocation, with requirements ranging from 1 to 5 units per task. This contrasts with simpler systems and necessitates more complex resource scheduling. The system features several repetitive tasks (INV_CHK, SHIP_CALC, CUST_NOTIF, WH_ALLOC, FINAL_PROC), indicating processes that require iterative execution or polling within a single transaction cycle.

5. Temporal Constraints and Dependency System

5.1 Mixed Precedence Model (Start-to-Start)

The ECOM-OPS-8T system implements a sophisticated temporal constraint system based on start-to-start precedence. This type of constraint specifies that a destination task can begin execution only after a given delay has elapsed since the start of the source task execution.

A critical feature of this system is the use of the `wait_all` flag, which modifies the behavior of constraints originating from repetitive tasks:

- * `wait_all: false`: The destination task can begin after the specified delay has elapsed from the start of the *first* execution of the source task. This allows for maximum parallelism.
- * `wait_all: true`: The destination task must wait for *all repetitions* of the source task to complete. The delay is then applied after the completion of the final repetition. This enforces a strict sequential integrity, ensuring a process is fully finished before its dependents can begin.

5.2 Dependency Matrix

Source Task	Destination Task	Delay (t.u.)	Type	Wait All
Payment_Validation (1)	Inventory_Check (2)	5	SS	false
Inventory_Check (2)	Shipping_Calculation (3)	9	SS	true
Payment_Validation (1)	Order_Confirmation (4)	21	SS	false
Inventory_Check (2)	Order_Confirmation (4)	16	SS	true
Shipping_Calculation (3)	Order_Confirmation (4)	10	SS	true
Order_Confirmation (4)	Customer_Notification (5)	8	SS	false
Shipping_Calculation (3)	Warehouse_Allocation (6)	16	SS	false
Payment_Validation (1)	Warehouse_Allocation (6)	10	SS	false
Order_Confirmation (4)	Warehouse_Allocation (6)	7	SS	true
Payment_Validation (1)	Quality_Check (7)	11	SS	true
Warehouse_Allocation (6)	Quality_Check (7)	10	SS	false
Shipping_Calculation (3)	Final_Processing (8)	13	SS	true
Payment_Validation (1)	Final_Processing (8)	11	SS	true
Order_Confirmation (4)	Final_Processing (8)	13	SS	false

Legend: SS = Start-to-Start

5.3 Critical Dependency Analysis

The dependency graph is complex, with multiple branching and convergence points. The process initiates primarily with **Payment_Validation (1)**, which triggers multiple parallel paths.

A key architectural feature is the role of **Order_Confirmation (4)** as a major synchronization point. It cannot start until payment is initiated (wait_all: false), but must wait for the *complete execution* of all repetitions of both **Inventory_Check (2)** and **Shipping_Calculation (3)** (wait_all: true). This creates a deliberate gate, ensuring that an order is only confirmed after all inventory and shipping checks are finalized, which is critical for business logic integrity.

The `wait_all` flag is used strategically to enforce process stages. For instance, the dependency from **Inventory_Check (2)** to **Shipping_Calculation (3)** is `wait_all: true`, meaning all inventory checks must be fully completed before

shipping can even be calculated. This suggests a business rule where shipping options may depend on the final, confirmed stock status.

In contrast, dependencies with `wait_all: false`, such as `Payment_Validation` (1) to `Inventory_Check` (2), allow the inventory process to begin as soon as the payment process starts, maximizing workflow parallelism in the early stages of the pipeline. The final steps, like `Final_Processing` (8), also exhibit this mixed-dependency pattern, highlighting a sophisticated orchestration of parallel and sequential execution.

6. Final Considerations and Recommendations

6.1 Architecture Summary

The ECOM-OPS-8T system is a powerful and complex architecture for business process automation. Its design reflects a sophisticated approach to managing high-volume, multi-stage workflows, balancing the need for speed (parallelism) with the need for process integrity (sequential gates).

The high resource availability (110 units), heterogeneous task requirements, and the strategic use of repetitive tasks and mixed `wait_all` constraints demonstrate a mature and highly configurable system designed for performance and reliability in a demanding e-commerce environment.

6.2 Key Architectural Features

- **Mixed-Constraint Model:** The strategic use of both `wait_all: true` and `wait_all: false` is a cornerstone of this architecture. It allows fine-grained control over the process flow, enabling parallelism where possible and enforcing strict sequential order where necessary for business rule compliance.
- **Heterogeneous Resource Allocation:** Unlike simpler systems, tasks have varied resource demands. This requires an advanced scheduler to optimize the allocation of the 110 available units and prevent resource starvation for critical tasks.
- **Centralized Synchronization Points:** The architecture relies on key tasks, particularly `Order_Confirmation` (4), to act as gates that synchronize multiple asynchronous upstream processes. This is a robust pattern for ensuring data and process consistency before proceeding to subsequent stages.

6.3 Knowledge Transfer Recommendations

For effective knowledge transfer, it is recommended to follow a structured training path that includes: understanding the global parameters and the business significance of the operational window, studying the service architecture and the role of each in the e-commerce pipeline, detailed analysis of individual tasks with emphasis on their heterogeneous resource and repetition characteristics, mapping the complex temporal dependency graph, paying special attention to the difference between `wait_all: true` and `wait_all: false` constraints, and simulating the operational flow to identify critical paths and potential bottlenecks, especially around the Order_Confirmation (4) synchronization point.

Appendix A: Glossary

Term	Definition
ECOM-OPS	E-commerce Operations - System domain identifier
t.u.	Time Unit - Base unit for time measurement in the system
r_max	Maximum number of computational resources allocable simultaneously
t_max	Maximum duration of operational window in time units
sig	Signature - A short, unique code identifying a task
res_q	Resource Quantity - The number of computational units a task requires
Start-to-Start (SS)	Type of precedence constraint based on task start rather than completion
wait_all	A constraint flag. If <code>true</code> , the destination task waits for all repetitions of the source task to finish. If <code>false</code> , it can start after a delay from the first execution of the source task.
Repetitive Task	Task configured to execute multiple repetitions ($rc > 0$)
rc	Repeat Count - Number of repetitions for a repetitive task
rd	Repeat Delay - Time delay between repetitions of a repetitive task
max_c	Maximum Concurrency - Maximum number of concurrent executions allowed for a task
Pipeline	A sequence of processing stages where the output of one stage is the input for the next.

Appendix B: Reference Documents

Code	Document Title	Version
REF-001	ECOM-OPS-8T System Requirements Specification	[To be filled]
REF-002	E-commerce Process Flow Specification	[To be filled]
REF-003	Verification and Validation Plan	[To be filled]
REF-004	Business Operations Manual	[To be filled]

END OF DOCUMENT